UNISPHERE: CAMPUS HUB

Product Requirement Document (PRD) v2.0

1. Executive Summary

UniSphere is a full-stack campus management and events discovery portal designed to centralize student extracurricular life, automate coordinator check-in workflows, and provide analytical telemetry dashboards. By combining modern databases, caching layers, and semantic artificial intelligence, the platform resolves common administrative pain points such as double-booked venues, poor student event engagement, and manual paper-based attendance registers.

This iteration details a complete technology migration. The database models, REST endpoint controllers, security guards, and seeding scripts have been ported to Node.js (TypeScript/Express). The frontend state container has been refactored onto Redux Toolkit. Advanced integrations are introduced, including simulated SMTP email traces, Swagger API documents, Redis caching, Cosine-Similarity RAG vector heuristics, and 8 highly detailed interactive drawers.

2. User Personas & Objectives

A. Student (Alex Rivera)

• **Profile:** Senior Computer Science major, Honors candidate seeking relevant technical seminars.

• **Pain Points:** Physical flyers on bulletin boards, isolated group chats, lost boarding ticket codes.

• **Goal:** Discover events matching profile tags, display digital check-in passes, request transcript packages, and review courses.

B. Faculty Coordinator (Dr. Sarah Jenkins)

• **Profile:** Assistant Professor of Data Science and student organization advisor.

• **Pain Points:** Resource double-bookings, manual attendance tracking, low participation rates.

• **Goal:** Publish non-conflicting event schedules, check in attendees via simulated QR code scanner, and evaluate engagement predictions.

C. Administrator (Admin Chief)

• **Profile:** Director of Student Life Operations.

• **Pain Points:** Cluttered manual approval pipelines, lack of diagnostics metrics, complex local setup.

• **Goal:** Moderate proposed clubs, verify server parameters, flush Redis caches, and run DB seed synchronization.

3. Product Features & Drawers

UniSphere implements 8 interactive sliding drawers in the user interface to simulate real-time university services and administrative dashboard options:

• **Announcements:** Renders campus news feeds (e.g. term registration schedules, sports tryouts) with pin priority filters.

• **Academic Calendar:** Interactive month grid. Selecting date filters display agenda items. Features a form to add custom task reminders.

• **Campus Directory:** List of campus members with status tags (Online, Offline, In Meeting). Features search query filtering.

• **Academic Record:** Displays cumulative CGPA (3.92) and completed credits. Includes a Request Transcript button with simulated email dispatch.

• **Course Catalog:** Searchable course database with professor names and syllabus details. Toggles simulated enrollment requests.

• **Faculty Hub:** Advisory office lookup directory. Includes a scheduled consultation booking scheduler.

• **Student Affairs:** Overview of housing, placements, and health clinics. Includes a support ticket submit form.

• **System Admin:** Renders server diagnostics (SQLite database, Redis fallback, CPU, RAM). Includes buttons to run seeder scripts and clear Redis caches.

---

4. AI & Intelligent Systems (RAG & Regression)

4.1 RAG Vector Similarity Search Heuristics

Calculated in real-time inside the AI Service. Student interests (e.g. computer science, machine learning, sports) and event description details are parsed, stop-words are filtered, and frequency term vectors are generated. The system calculates the Cosine Similarity coefficient (the dot product of both vectors divided by their magnitudes) to rank event matches on the student dashboard, complete with justification strings.

4.2 Attendance Regression Predictor

Uses historical attributes (event categories, capacities, and day of week) to predict attendance rates and occupancy limits on the coordinator dashboard, identifying factors contributing to registration likelihood.

4.3 Smart Schedule Suggestion

Conflict optimization suggestions identify optimal dates and times with the lowest potential conflict index based on current calendars and coordinator workloads.

5. Technical Architecture & Database Schemas

5.1 Express API Backend & Dual-Mode DB

Built with Express (Node.js/TypeScript) utilizing Sequelize ORM. The database connects to PostgreSQL in production and falls back to zero-config SQLite (`unisphere.sqlite`) in dev mode for offline ease of use.

5.2 Caching Strategy (Redis)

Event schedules are cached in Redis to minimize database queries. Cache is invalidated on event creation, deletion, or admin approval. Fallback to in-memory bypass is active if Redis is offline.

5.3 System Data Models

- **Users:** id (PK), name, email, password, role (STUDENT/FACULTY/ADMIN), department, profileImage
- **Clubs:** id (PK), name, description, bannerImage, creatorId, membersCount, status (PENDING/ACTIVE)
- **Events:** id (PK), title, description, date, time, location, campus, maxCapacity, status (PENDING/APPROVED/REJECTED), bannerImage, category, clubId, coordinatorId, engagementScore
- **Registrations:** id (PK), eventId, studentId, status (REGISTERED/CANCELLED), passCode
- **Attendances:** id (PK), eventId, studentId, checkedInAt, checkedById
- **Notifications:** id (PK), userId, title, message, type, isRead
- **Feedbacks:** id (PK), eventId, studentId, rating (1-5), comment

---

6. Quality Assurance, CI/CD, & Cloud Blueprint

6.1 Testing Suite

Configured unit tests in package.json (run via npm test) which assert JWT tokenization, Cosine Similarity calculations, and prediction regression models.

6.2 CI/CD Pipelines

Provided a GitHub Actions CI/CD configuration (.github/workflows/ci.yml) which automatically triggers dependencies installations, TypeScript compiler builds, and unit test suites on pushes and pull requests.

6.3 Cloud & Multi-Container Blueprints

Provided docker-compose.yml files linking PostgreSQL database, Redis cache, Node backend, and Nginx frontend client, alongside Render render.yaml blueprints and Vercel routing configs.